

Problem: We have n items. Each item j has a value v_j and weight w_j . The bag has a maximum weight W . What items can you take to maximize the total value of all the items taken such that the total weight of the items taken is at most W ? (Note: It is not possible to put a part of an item into the bag)

- In order to explain the structure of the problem, it is better to provide an example:

Input:

- Capacity of knapsack $W=40$
- List of item values(v_j): [10, 30, 20, 40, 20]
- Weights of items (w_j): [30, 10, 40, 20, 15]

Possible Solutions:

- [*Item1*, *Item2*] with total value of 40
- [*Item3*] with total value of 20
- [*Item4*, *Item5*] with total value of 60

Optimal Solution: [*Item2*, *Item4*] with total value of 70

Dynamic programming exploits all possible solutions in a systematic way.

To solve this question, let's use the guideline from the assignment. You do not have to follow the guideline. You can write the pseudo-code, prove the optimal substructure, and calculate the run time.

Here are guided steps to help you while thinking about a solution:

- Identify the “last” question you need to answer in developing a solution.
 - Do we place the last item in the knapsack?
 - for the last item, we either put it in, or not, so there are two options for each decision.
- Define subproblems, express the solution to the overall problem in terms of the subproblems.
 - Subproblems: $T[i, j]$ to be the maximum possible value of a knapsack with capacity j and can use first i items.

Then the optimal solution will be evaluated when $i = n$ and $j = W$ which will evaluate $T[n, W]$

(iii) Define and prove optimal substructure. (Using induction)

In our problems, we used exchange of arguments to prove the optimal substructure. In this example, I wanted to provide induction proof.

- Our Inductive Hypothesis is that we compute all $T[i, j]$ s correctly for all pairs of $(i, j) < (u, v)$. Here $<$ means (i, j) is computed before (u, v) .
- The base cases for this induction are the base cases for our algorithm, which are $T[i, 0] = T[0, j] = 0$. They are trivially correct.
- Suppose $T[i-1, j]$ and $T[i-1, j-w_i]$ store the maximum value possible on items $1, 2, \dots, i-1$ and the given weight.
- For item i there are two choices: take it or don't take it.
- If we do not take it, the value is equivalent to $T[i-1, j]$ since we know that it is optimal for $i-1$.
- If we take the item, the bag can only fit $j - w_i$ more weight. This means the value is $T[i-1, j - w_i] + v_i$ since we know that it is optimal for $i-1$. Therefore the recurrence is correct.

(iv) Formulate a recursive solution to the subproblems. Do not forget to specify the base case(s).

For $1 \leq i \leq m$ and $1 \leq j \leq n$:

$$T[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ (no items left)} \\ 0 & \text{if } j = 0 \text{ (no capacity left)} \\ T[i-1, j] & \text{item } i \text{ is not taken} \\ T[i-1, j - w_i] + v_i & \text{if } j \geq w_i \text{ (item } i \text{ is taken)} \end{cases}$$

(v) Provide the pseudo code of the bottom-up procedure you use to compute the value of the optimal solution, as well as the procedure for reconstructing the optimal solution.

Algorithm 1 For a capacity W , a list of values V , number of items n , a list of weights wt , and an empty list to store the results $T[n][W]$

```

1: procedure KNAPSACK01( $W, V[], n, wt[], T[n][W]$ )
2:   for  $i = 0$  to  $n$  do
3:     for  $j = 0$  to  $W$  do
4:       if  $i==0$  or  $j==0$  then
5:          $T[i, j] = 0$ 
6:       else
7:         if  $wt[i - 1] \leq j$  then
8:            $T[i, j] = MAX(V[i - 1] + T[i - 1, j - wt[i - 1]], T[i - 1, j])$ 
9:         else
10:           $T[i, j] = T[i - 1, j]$ 
11:        end if
12:      end if
13:    end for
14:  end for
15:   $res = T[n, W]$  ▷ maximum value in the knapsack
16:  return  $res, T$  ▷ return the maximum value and result table
17: end procedure

```

Algorithm 2 For a capacity W , a list of values V , number of items n , a list of weights wt , and a list storing the results $T[n][W]$

```

1: procedure PRINTKNAPSACK( $W, V[], n, wt[], T[] []$ )
2:    $res = T[n][W]$  ▷ maximum value for the knapsack
3:    $w = W$  ▷ the knapsack capacity
4:   for  $i = n$  down to  $0$  do
5:     if  $i > 0 \ \&\& \ res > 0 \ \&\& \ res == K[i-1][w]$  then
6:       continue
7:     else
8:       Print  $wt[i-1]$  ▷ This item is included in the result
9:        $res = res - V[i-1]$  ▷ Since the item is included, subtract the value
10:       $w = w - wt[i-1]$  ▷ Since the item is included, subtract the weight
11:    end if
12:  end for
13: end procedure

```

(vi) Characterize the runtime of the resulting procedure assuming that you would implement your solution using a **bottom-up procedure**.

- Number of subproblems: $\theta(n)$ since $1 \leq i \leq n$
- Number of options for each subproblem: j is $\theta(W)$ since $1 \leq j \leq W$
- Cost of evaluation of each option: $\theta(1)$ since this is basic comparison
- $T(n) = \text{Num. of subproblems} \times \text{Num. of options for each subproblem} \times \text{Cost of evaluation}$
- $T(n) = \theta(n) \times \theta(W) \times \theta(1)$
- $T(n) = \theta(nW)$